THE NEW STACK WEBINAR
Design Retrieval as a Funnel
See how multi-stage retrieval beats retrieve-everything-then-rerank at scale — with real tradeoffs.
Register Now

Introducing ShipAI by TDS

ARTIFICIAL INTELLIGENCE

Hitchhiker's Guide to RAG with ChatGPT API and LangChain

Build a simple Python RAG pipeline using your local files as context

Maria Mouschoutzi
June 26, 2025 • 6 min read

Hanna Barakat & Archival Images of AI + AIxDESIGN / https://betterimagesofai.org / https://creativecommons.org/licenses/by/4.0/

Nowadays, LLMs can easily generate tons of words and responses based on general knowledge, but what happens when we need answers requiring accurate and specific knowledge? Solely generative models frequently struggle to provide answers on domain-specific questions for a bunch of reasons; maybe the data they were trained on is now outdated, maybe what we are asking for is *really* specific and specialized, maybe we want responses that take into account personal or corporate data that just aren't public… 🤷‍♀️ the list goes on.

So, how can we leverage generative AI while keeping our responses accurate, relevant, and down-to-earth? A good answer to this question is the Retrieval-Augmented Generation (RAG) framework. RAG is a framework that consists of two key components: retrieval and generation (duh!). Unlike solely generative models that are pre-trained on specific data, RAG incorporates an extra step of retrieval that allows us to push additional information into the model from an external source, such as a database or a document. To put it differently, a RAG pipeline allows for providing coherent and natural responses (provided by the generation step), which are also factually accurate and grounded in a knowledge base of our choice (provided by the retrieval step).

> *🍨**DataCream** is a newsletter offering stories and tutorials on AI, data, tech. If you are interested in these topics, **subscribe here**.*

In this way, RAG can be an extremely valuable tool for applications where highly specialized data is needed, such as for instance customer support, legal advice, or technical documentation. One typical example of a RAG application is customer support chatbots, answering customer issues based on a company's database of support documents and FAQs. Another example would be complex software or technical products with extensive troubleshooting guides. One more example would be legal advice — a RAG model would access and retrieve custom data from law libraries, previous cases, or firm guidelines. The examples are really endless; however, in all these cases, the access to external, specific, and relevant to the context data enables the model to offer more precise and accurate responses.

Learn this step by step with the interactive **AI Engineer** roadmap.

So, in this post, I walk you through building a simple RAG pipeline in Python, utilizing ChatGPT API, LangChain, and FAISS.

What about RAG?

From a more technical perspective, RAG is a technique used to enhance an LLM's responses by injecting it with additional, domain-specific information. In essence, RAG allows for a model to also take into account additional external information — like a recipe book, a technical manual, or a company's internal knowledge base — while forming its responses.

This is very important because it allows us to eliminate a bunch of problems inherent to LLMs, as for instance:

- Hallucinations — making things up
- Outdated information — if the model wasn't trained on recent data
- Transparency — not knowing where responses are coming from

To make this work, the external documents are first processed into vector embeddings and stored in a vector database. Then, when we submit a prompt to the LLM, any relevant data is retrieved from the vector database and passed to the LLM along with our prompt. As a result, the response of the LLM is formed by considering both our prompt and any relevant information existing in the vector database in the background. Such a vector database can be hosted locally or in the cloud, using a service like Pinecone or Weaviate.

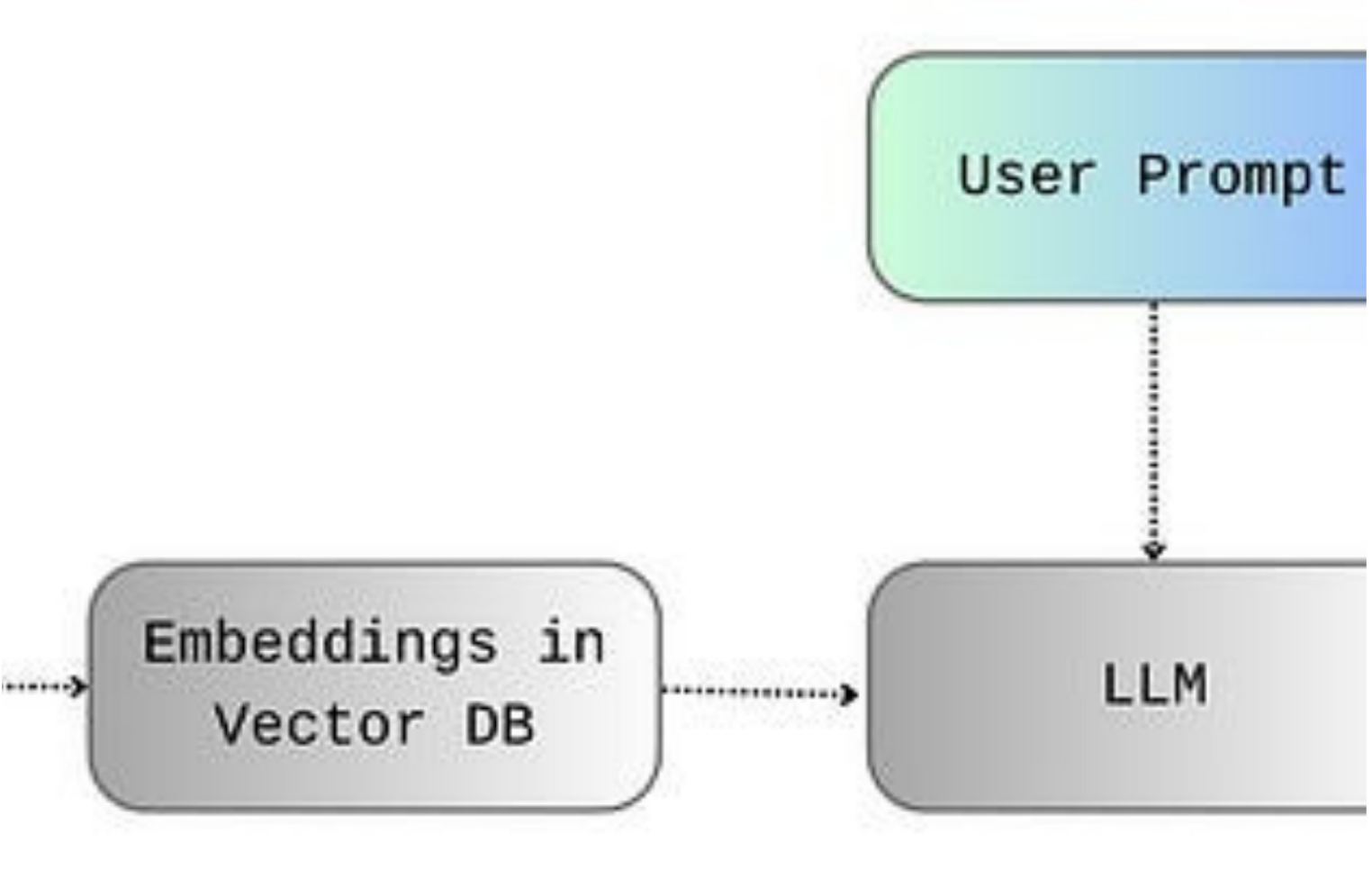


Image by author

What about ChatGPT API, LangChain, and FAISS?

The first component for building a RAG pipeline is the LLM model that will generate the responses. This can be any LLM, like Gemini or Claude, but in this post, I will be using OpenAI's ChatGPT models via their API platform. In order to use their API, we need to sign in and obtain an API key. We also need to make sure the respective Python libraries are installed.

```python
pip install openai
```

The other major component of building a RAG is processing external data — generating embeddings from documents and storing them in a vector database. The most popular framework for performing such a task is LangChain. In particular, LangChain allows:

- Load and extract text from various document types (PDFs, DOCX, TXT, etc.)
- Split the text into chunks suitable for generating the embeddings
- Generate vector embeddings (in this post, with the assistance of OpenAI's API)
- Store and search embeddings via vector databases like FAISS, Chroma, and Pinecone

We can easily install the required LangChain libraries by:

```python
pip install langchain langchain-community langchain-openai
```

In this post, I'll be using LangChain together with FAISS, a local vector database developed by Facebook AI Research. FAISS is a very lightweight package, and is thus appropriate for building a simple/small RAG pipeline. It can be easily installed with:

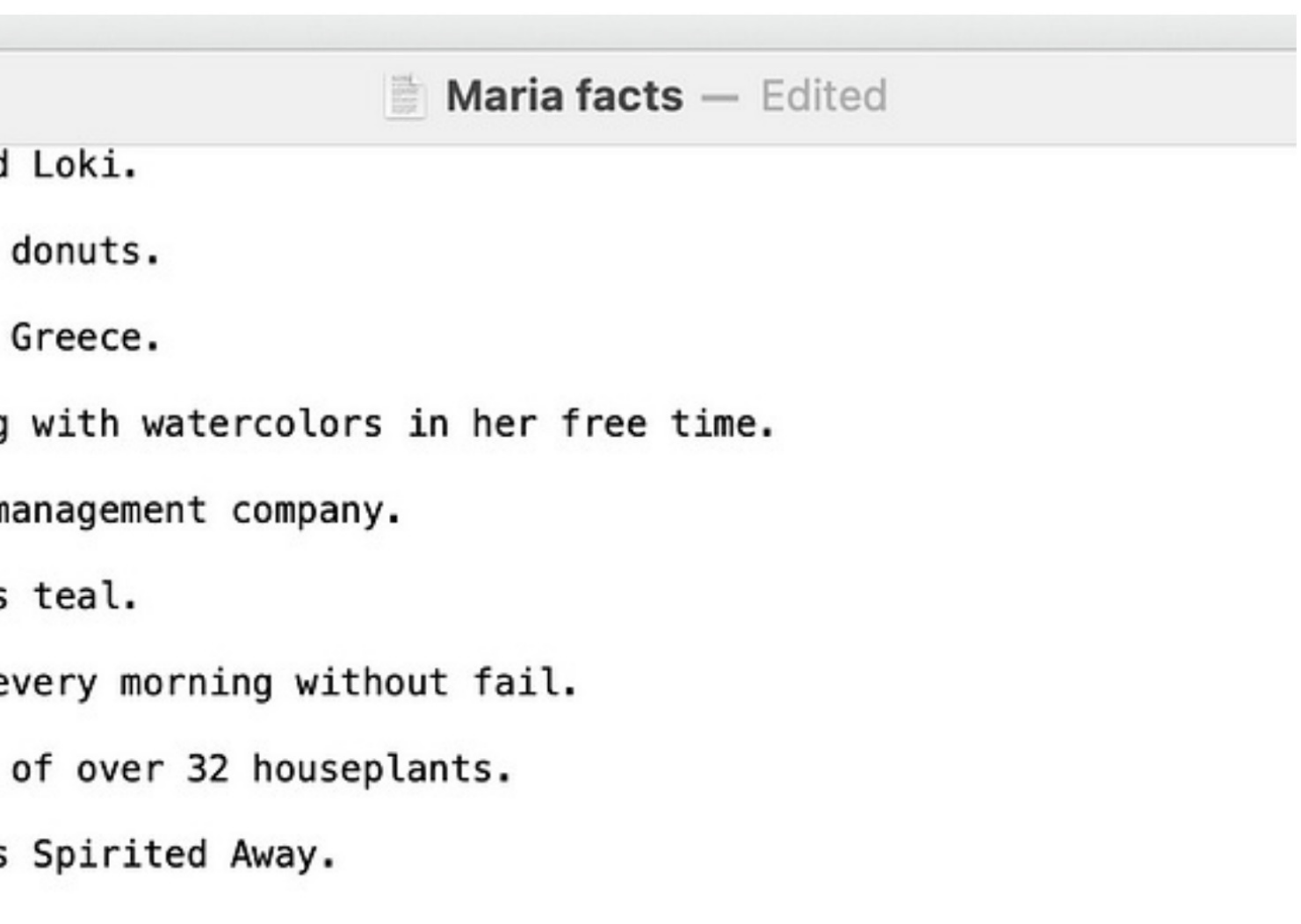
```python
pip install faiss-cpu
```

Putting everything together

So, in summary, I will use:

- ChatGPT models via OpenAI's API as the LLM
- LangChain, along with OpenAI's API, to load the external files, process them, and generate the vector embeddings
- FAISS to generate a local vector database

The file that I will be feeding into the RAG pipeline for this post is a text file with some facts about me. This text file is located in the folder 'RAG files'.



```
ed Loki.
s donuts.
, Greece.
ng with watercolors in her free time.
 management company.
is teal.
 every morning without fail.
n of over 32 houseplants.
is Spirited Away.
```

Now we are all set up, and we can start by specifying our API key and initializing our model:

```python
from openai import OpenAI # Chat_GPT API key api_key = "your key"

# initialize LLM
llm = ChatOpenAI(openai_api_key=api_key, model="gpt-4o-mini", temperature=0.3)
```

Then we can load the files we want to use for the RAG, generate the embeddings, and store them as a vector database as follows:

```python
# loading documents to be used for RAG
text_folder = "rag_files"

all_documents = []
for filename in os.listdir(text_folder):
    if filename.lower().endswith(".txt"):
        file_path = os.path.join(text_folder, filename)
        loader = TextLoader(file_path)
        all_documents.extend(loader.load())

# generate embeddings
embeddings = OpenAIEmbeddings(openai_api_key=api_key)

# create vector database w FAISS
vector_store = FAISS.from_documents(documents, embeddings)
retriever = vector_store.as_retriever()
```

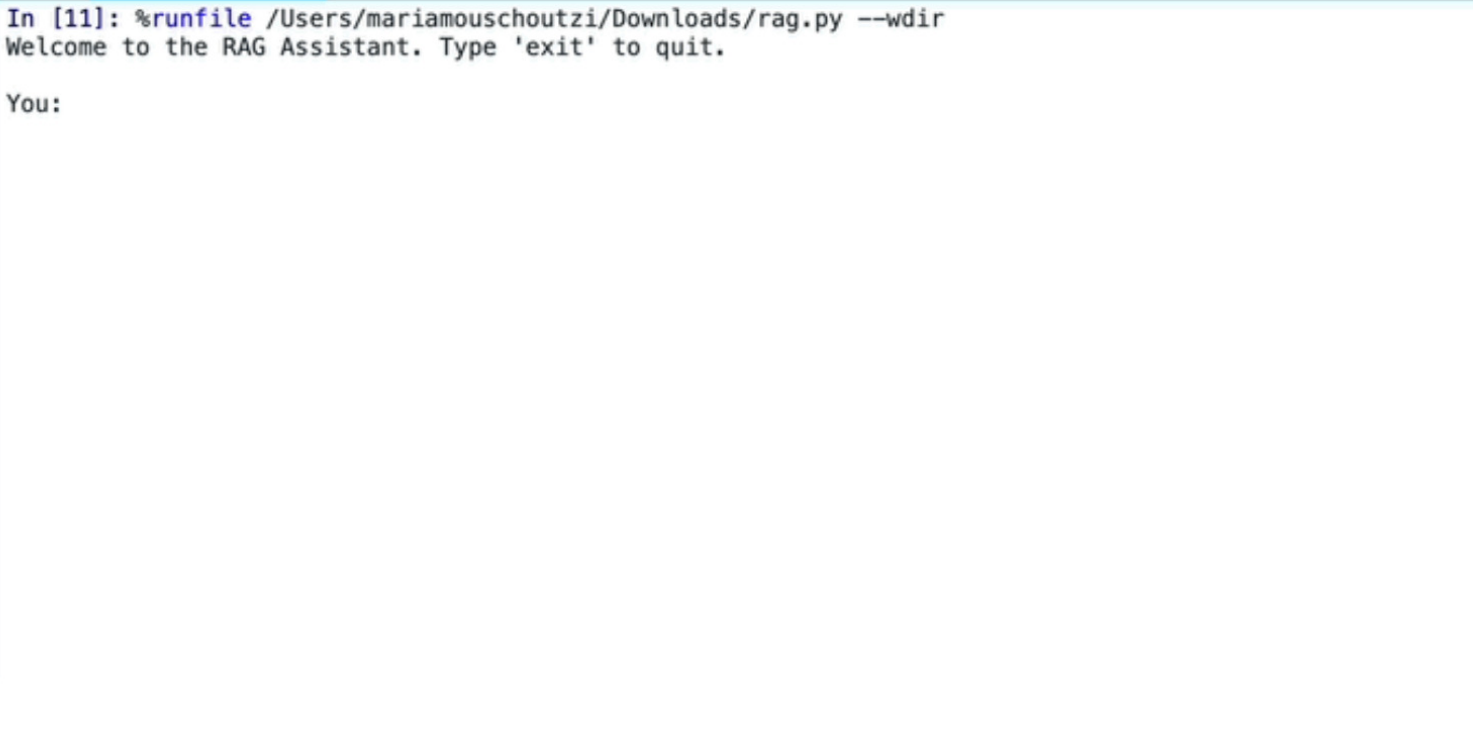
Finally, we can wrap everything in a simple executable Python file:

```python
def main():
    print("Welcome to the RAG Assistant. Type 'exit' to quit.\n")

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            print("Exiting…")
            break

        # get relevant documents
        relevant_docs = retriever.get_relevant_documents(user_input)
        retrieved_context = "\n\n".join([doc.page_content for doc in relevant_docs])

        # system prompt
        system_prompt = (
            "You are a helpful assistant. "
            "Use ONLY the following knowledge base context to answer the user. "
            "If the answer is not in the context, say you don't know.\n\n"
            f"Context:\n{retrieved_context}"
        )

        # messages for LLM
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_input}
        ]

        # generate response
        response = llm.invoke(messages)
        assistant_message = response.content.strip()
        print(f"\nAssistant: {assistant_message}\n")

if __name__ == "__main__":
    main()
```

Notice how the system prompt is defined. Essentially, a system prompt is an instruction given to the LLM that sets the behavior, tone, or constraints of the assistant before the user interacts. For example, we could set the system prompt to make the LLM provide responses like talking to a 4-year-old or a rocket scientist — here we ask to provide responses only based on the external data we provided, the '*Maria facts*'

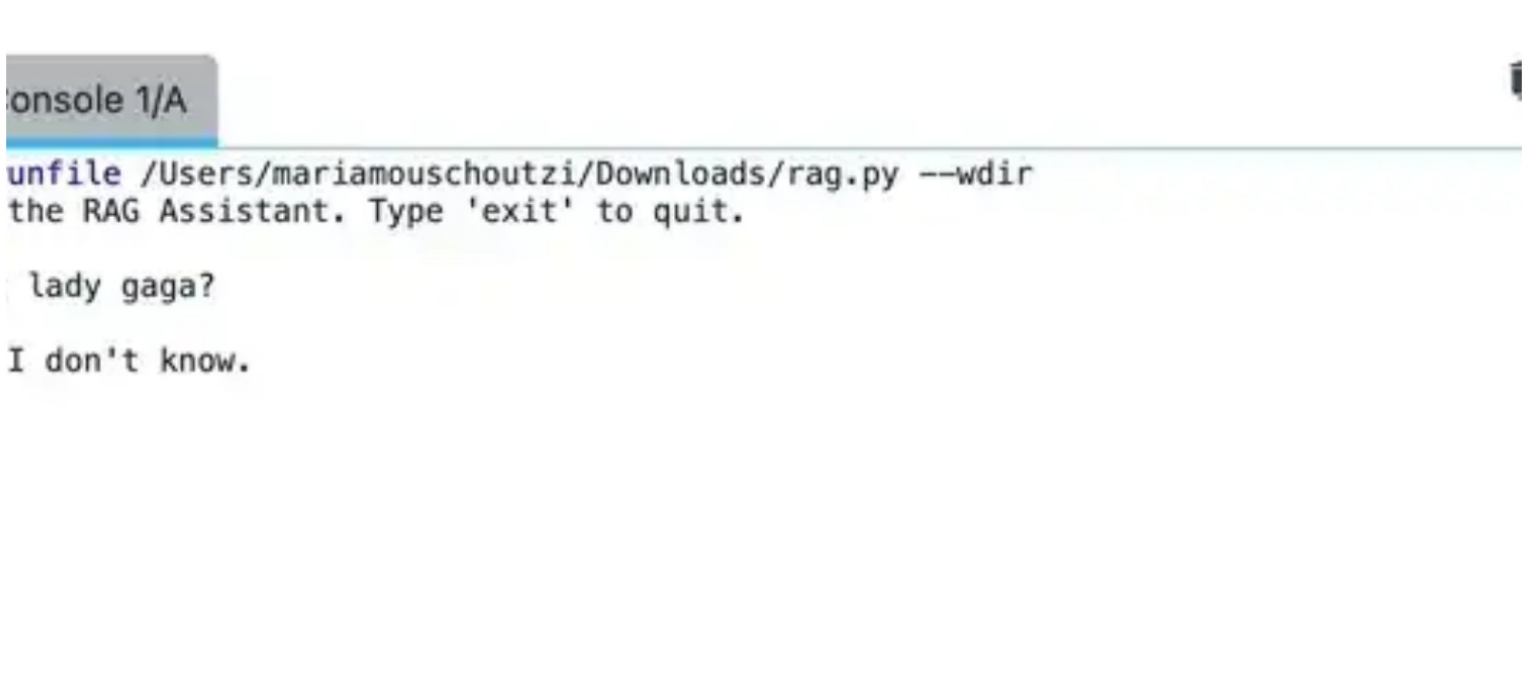
So, let's see what we've cooked! 🍳

Firstly, I ask a question that is irrelevant to the provided external datasource, to make sure that the model only uses the provided datasource when forming the responses and not general knowledge.



```
In [11]: %runfile /Users/mariamouschoutzi/Downloads/rag.py --wdir
Welcome to the RAG Assistant. Type 'exit' to quit.

You:
```

… and then I asked some questions specifically from the file I provided…



```
unfile /Users/mariamouschoutzi/Downloads/rag.py --wdir
the RAG Assistant. Type 'exit' to quit.

 lady gaga?

I don't know.
```

✨✨✨✨

On my mind

Apparently, this is a very simplistic example of a RAG setup — there's much more to consider when implementing it in a real business environment, such as security concerns around how data is handled, or performance issues when dealing with a larger, more realistic knowledge corpus and increased token usage. Nonetheless, I believe OpenAI's API is truly impressive and offers immense, untapped potential for building custom, context-specific AI applications.

✨Thank you for reading!✨

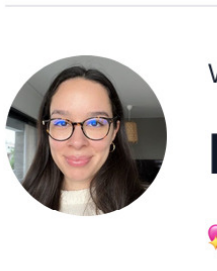
• • •

*If you made it this far, **you might find pialgorithms useful** — a platform we've been building that helps teams securely manage organizational knowledge in one place.*

• • •

Loved this post? Let's be friends! Join me on:

📰***Substack*** 💌 ***Medium*** 💼***LinkedIn*** ☕***Buy me a coffee!***



WRITTEN BY
Maria Mouschoutzi
💖 all things data, AI/ML, tech 🙃 30 @Athens, GR

See all from Maria Mouschoutzi

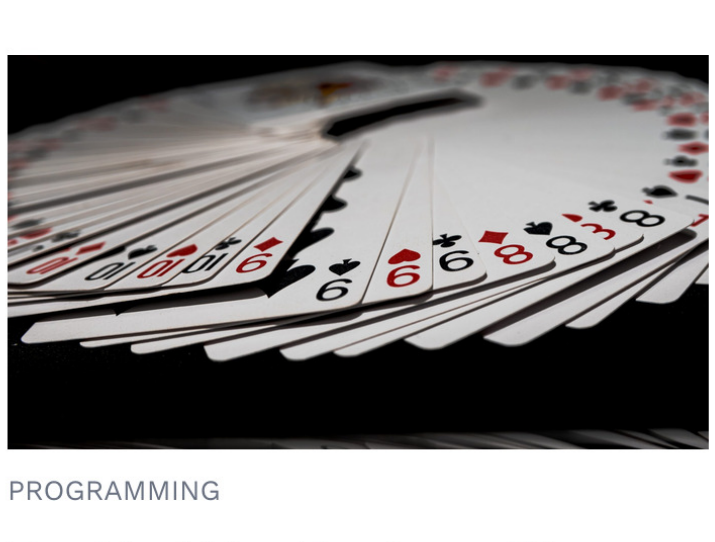
ChatGPT | Data Science | Langchain | Python | Retrieval Augmented

Share This Article

Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

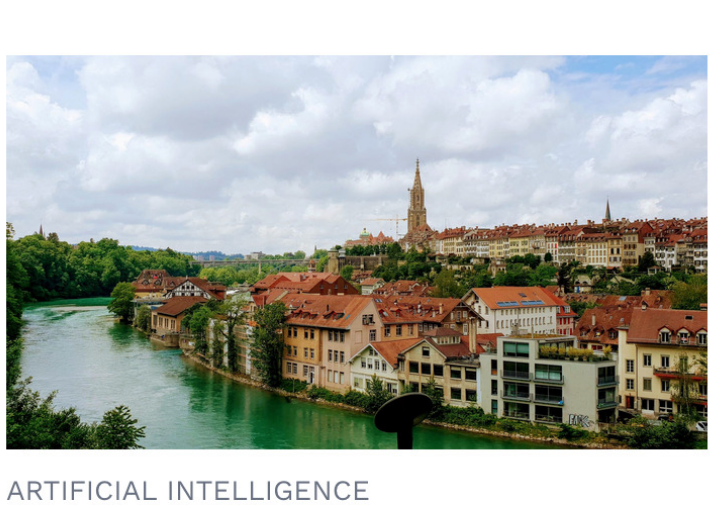
Related Articles



PROGRAMMING
Beating the Dealer with Simple Statistics
Simulating Thousands of Blackjack Card Counting Strategies with Data Science Tools
Marco Santos
September 7, 2019 • 8 min read

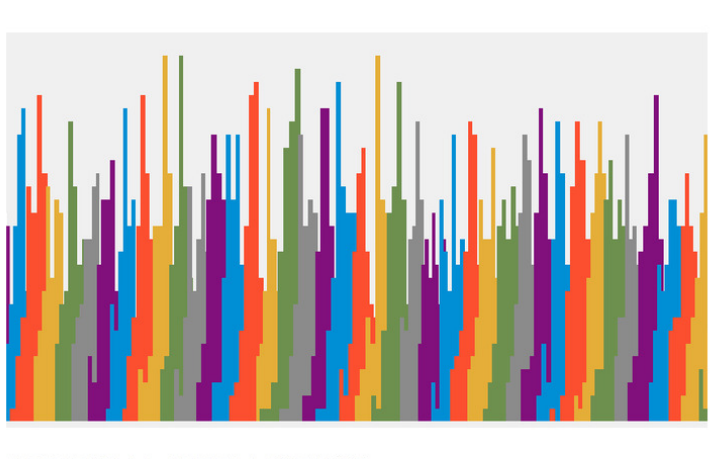
ARTIFICIAL INTELLIGENCE
Building Machine Learning Operations for Businesses
A Blueprint for Effective MLOps to Support Your AI Strategy
John Adeojo
June 20, 2023 • 5 min read

MACHINE LEARNING
Beware of Halo Effect in Data Science Projects
Do you only gather pieces of information that support your hypothesis?
Sue Lynn
January 31, 2022 • 5 min read



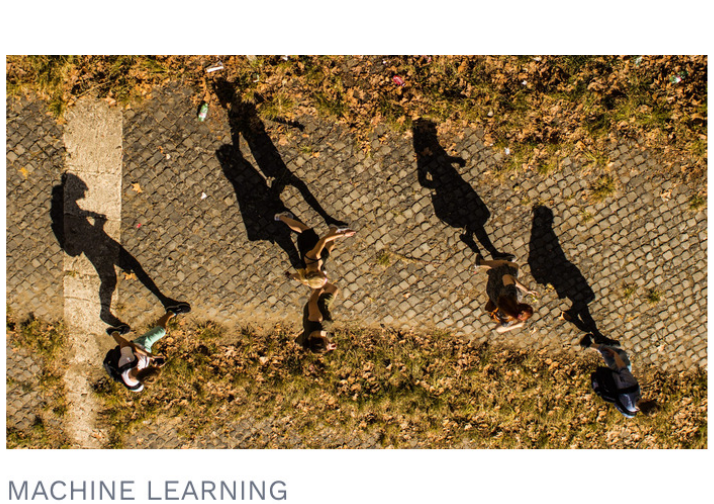
ARTIFICIAL INTELLIGENCE
Understanding Fast-RCNN for Object Detection
The fast-RCNN paper highlights the drawbacks of SPPNet and RCNN, and build a relatively fast and accurate model
Parth Rajesh Dedhia
November 6, 2020 • 8 min read

MACHINE LEARNING
Farm to Table: The Workflow of a Classification Model
Comparing logistic regression and random forest classification in recipe recommendations
John Lenehan
April 2, 2024 • 18 min read



ARTIFICIAL INTELLIGENCE
Conducting Bayesian Inference in Python using PyMC3
Revisiting the coin example and using PyMC3 to solve it computationally.
Robert Kübler
December 23, 2020 • 9 min read

ARTIFICIAL INTELLIGENCE
Neural Machine Translation (NMT) with Attention Mechanism
A guide to translate languages with Deep Learning!
Harshil Patel
June 5, 2020 • 12 min read



MACHINE LEARNING
Too Many Features? Let's Look at Principal Component Analysis
Home-brewed machine learning model series
Hector Mejia
June 24, 2023 • 13 min read

ARTIFICIAL INTELLIGENCE
Increase Llama 2's Latency and Throughput Performance by Up to 4X
Real-world benchmarks for Llama-2 13B
Het Trivedi
August 9, 2023 • 7 min read

© Insight Media Group, LLC 2026